c13n #105

c13n

2026 年 11 月 5 日

第 I 部

编译器中的指令选择与寄存器分配

叶家炜

Oct 31, 20

在现代编译器后端中，指令选择与寄存器分配构成了代码生成阶段的两个核心环节。它们共同决定了最终生成代码的性能、代码大小以及功耗表现。指令选择负责将编译器中间表示（IR）映射到目标处理器的指令集，而寄存器分配则将程序中的虚拟寄存器映射到有限的物理寄存器资源上。两者并非孤立存在，而是相互交织、相互影响，共同塑造着最终的机器码质量。

以 ARM 和 x86 架构为例，相同的中间表示在不同目标机上的代码生成结果可能存在显著差异。在某些计算密集型场景下，ARM 平台由于指令编码更紧凑，往往能生成更小的代码体积，而 x86 平台则可能因其丰富的复杂指令而展现出更优的性能表现。这种差异不仅源于指令集架构本身的设计哲学，也反映了指令选择算法在模式覆盖和代价建模上的不同取舍。

1 指令选择

1.1 从三地址 IR 到目标机指令的映射

指令选择的核心任务是将编译器前端生成的三地址中间表示转换为目标处理器能够直接执行的机器指令。在三地址 IR 中，每条指令通常形如 `a = b op c`，这种形式便于进行各种优化变换，但在映射到实际硬件指令时需要考虑诸多约束条件。

以 LLVM 的 SelectionDAG 为例，一个简单的加法操作在 IR 层面可能表示为 `add i32 %a, %b`，而在目标机指令选择阶段，这个操作可能被映射为一条 `ADD` 指令，或者在某些架构上被分解为多条更简单的指令。映射过程不仅要保证语义等价，还要考虑指令延迟、吞吐量以及代码大小等因素。

1.2 模式覆盖与代价模型

模式覆盖是指指令选择算法能够识别并匹配的目标机指令模式集合。现代处理器通常提供多种执行相同功能的指令变体，例如普通的算术指令与融合乘加指令（FMA）。算法需要在这些变体中做出选择，这就需要一个精确的代价模型来量化不同选择的优劣。

代价模型通常综合考虑指令执行周期、寄存器压力、代码密度等多个维度。在异构指令集如 RISC-V 向量扩展（RVV）中，代价模型还需要处理向量长度可变带来的复杂性，因为相同的逻辑操作可能对应不同长度的向量指令，其性能表现差异巨大。

1.3 树覆盖算法的实现原理

树覆盖算法是指令选择中最经典的方法之一，其代表性实现包括 BURG 及其变体。该算法将 IR 表示构建为树形结构，然后通过自底向上的动态规划过程寻找最优的指令覆盖方案。

在 BURG 算法中，状态机生成阶段会为每种可能的 IR 节点类型预先计算最优的指令匹配方案。动态规划阶段则利用这些预计算结果，高效地为整个表达式树找到全局最优的指令序列。这种方法的优势在于计算效率高，但受限于树形结构的表达能力，难以处理具有多输出的复杂指令模式。

1.4 图覆盖算法的扩展能力

相比树覆盖算法，基于图的覆盖方法能够处理更复杂的指令模式，特别是那些具有多个输出或复杂约束的指令。PBQP（Partitioned Boolean Quadratic Programming）是这类方法的典型代表，它将指令选择建模为一个二次规划问题，通过求解该问题来获得最优的指令覆盖方案。

图覆盖方法特别适合处理捆绑约束（Tied Constraints），即某些指令要求特定操作数必须位于同一物理寄存器中。这种约束在 SIMD 指令和某些特殊功能单元中非常常见，传统的树覆盖方法难以有效处理这类场景。

1.5 LLVM GlobalISel 的演进

LLVM 的 GlobalISel 代表了指令选择技术的最新发展方向。与传统的 SelectionDAG 相比，GlobalISel 采用了更加模块化的设计，将指令选择过程分解为多个独立的阶段：IR 翻译、合法化、指令选择和寄存器组分配。

这种分阶段的设计使得 GlobalISel 能够更好地支持异构指令集和新型处理器架构。在处理 RISC-V 向量扩展时，GlobalISel 可以通过独立的合法化阶段处理向量长度不匹配的问题，而不需要在指令选择阶段处理所有可能的组合情况。

1.6 GCC 的生成式方法

GCC 采用了不同的技术路线，其 genautomata 工具能够根据目标机的指令描述自动生成指令选择状态机。这种方法将指令集的描述与选择算法的实现分离，使得支持新架构的工作更加系统化。

peephole2 优化是 GCC 指令选择的另一重要组成部分，它在指令选择之后进行局部模式匹配，能够识别并优化某些跨越多条指令的复杂模式。这种两阶段的方法在保持整体效率的同时，也提供了足够的灵活性来处理目标机的特殊指令。

2 寄存器分配

2.1 活跃区间与干涉关系

寄存器分配问题的核心在于如何将程序中的无限虚拟寄存器映射到有限的物理寄存器集合。活跃区间分析是解决这一问题的关键步骤，它确定了每个变量在程序执行过程中的活跃范围。

活跃区间的概念可以用数学方式表达：对于变量 v，其活跃区间可以表示为 $I(v) = [start(v), end(v)]$，其中 $start(v)$ 和 $end(v)$ 分别表示变量首次定义和最后使用的程序点。当两个变量的活跃区间存在重叠，即 $I(v1) \cap I(v2) \neq \emptyset$ 时，它们之间存在干涉关系，不能分配到同一物理寄存器。

2.2 图染色分配器的经典流程

Chaitin-Briggs 图染色算法是寄存器分配中最具影响力的方法之一。该算法将寄存器分配问题建模为图染色问题，其中变量对应图中的顶点，干涉关系对应边，物理寄存器对应颜色。

算法包含五个主要阶段：简化（Simplify）阶段移除低度数顶点并将其压入栈中；合并（Coalesce）阶段尝试合并具有拷贝关系的变量以减少寄存器需求；冻结（Freeze）阶段处理无法合并但也不需要溢出的变量；溢出（Spill）阶段为无法染色的变量选择溢出位置；选择（Select）阶段从栈中弹出变量并为其分配颜色。

2.3 线性扫描分配器的效率优势

Poletto 和 Sarkar 提出的线性扫描算法在编译时间效率方面具有显著优势。与图染色算法的 $O(n^2)$ 复杂度相比，线性扫描算法能够在 $O(n \log n)$ 时间内完成分配，这对于 JIT 编译器等对编译时间敏感的应用场景尤为重要。

线性扫描算法的核心思想是按照活跃区间的起始位置对变量进行排序，然后维护一个按结束位置排序的活跃集合。当处理新变量时，算法首先尝试为其分配已释放的寄存器，如果没有可用寄存器则选择结束位置最晚的变量进行溢出。

2.4 LLVM 贪心分配器的现代特性

LLVM 的贪心寄存器分配器在传统线性扫描基础上进行了大量改进，引入了基于回合的分配策略和活跃区间拆分机制。这种方法能够更好地处理复杂的寄存器约束和目标机特定的分配需求。

在处理寄存器组约束时，LLVM 贪心分配器会考虑不同指令对寄存器组的要求。例如，某些 ARM NEON 指令要求操作数位于特定的寄存器组中，分配器需要在满足这些约束的前提下最小化溢出和拷贝操作。

2.5 寄存器压力与指令调度的协同

寄存器分配与指令调度之间存在复杂的相互作用。过度的指令重排可能导致寄存器压力急剧上升，而保守的调度策略又可能无法充分利用指令级并行性。现代编译器通常采用启发式方法来平衡这两个目标。

寄存器压力估计算法需要考虑循环嵌套深度、基本块边界以及函数调用等因素。在软件流水线场景中，模变量展开（Modulo Variable Expansion）技术通过展开循环来增加可用的寄存器空间，从而缓解寄存器压力。

3 协同优化策略

3.1 传统分离模型的局限

传统编译器将指令选择和寄存器分配作为两个独立的阶段顺序执行，这种分离模型虽然简化了实现复杂度，但可能导致次优的代码生成结果。指令选择阶段可能选择需要大量临时寄存

器的复杂指令，而这些指令在后续的寄存器分配阶段可能导致大量溢出操作。
一个典型的例子是融合乘加指令的选择。虽然 FMA 指令在计算性能上具有优势，但它要求三个操作数同时可用，这可能在寄存器分配阶段造成额外的压力。如果编译器在指令选择阶段能够预见这种压力，可能会选择保守的分离乘加指令序列。

3.2 带寄存器压力的指令选择

现代编译器开始探索在指令选择阶段就考虑寄存器压力的方法。这种方法需要修改代价模型，将寄存器分配的预期开销纳入考虑范围。
具体实现中，指令选择算法会为每种候选指令模式计算一个综合代价，包括指令本身的执行代价以及预期导致的寄存器分配开销。这种方法虽然增加了指令选择的计算复杂度，但能够生成整体更优的代码。

3.3 基于约束的指令重写

重物化（Rematerialization）是一种重要的协同优化技术，它通过重新计算某些值来避免寄存器溢出。当一个值可以在指令选择阶段被识别为可重物化时，寄存器分配器可以选择溢出该值而不是保留其在寄存器中的副本。
重物化决策需要在指令选择和寄存器分配之间建立反馈机制。指令选择阶段需要标记可重物化的值，而寄存器分配阶段则根据实际的寄存器压力情况决定是否应用重物化。

4 工程实践与调试

4.1 编译器内部状态的查看

现代编译器提供了丰富的调试选项来帮助开发者理解指令选择和寄存器分配的过程。在 LLVM 中，`llc -print-after-all` 选项可以输出编译过程中的所有中间状态，包括指令选择前后的 IR 表示和寄存器分配的结果。
GCC 提供了类似的 `-fdump-rtl-*` 选项，可以输出 RTL（Register Transfer Language）各个阶段的表示。这些工具对于定位指令选择失败或寄存器分配异常等问题非常有用。

4.2 可视化分析工具

活跃区间的可视化对于理解寄存器分配行为具有重要价值。开发者可以使用 graphviz 等工具将活跃区间关系绘制为干涉图，从而直观地观察变量间的干涉关系和分配结果。
交互式 Web 界面可以进一步提升分析效率，允许开发者动态调整分配参数并观察结果变化。这种方法特别适合教学和研究用途，能够帮助理解复杂的分配算法行为。

4.3 典型问题的诊断方法

过早溢出和指令未匹配是两种常见的后端问题。过早溢出通常表现为寄存器分配阶段产生了不必要的内存访问，而指令未匹配则表现为某些优化机会被错过。
诊断这些问题时，开发者可以对比不同编译选项下的汇编输出，或者使用目标机的性能计数器来量化问题的影响程度。汇编 diff 脚本结合 `objdump` 工具可以自动化地识别代码生成

差异。

4.4 测试与回归保障

寄存器分配和指令选择算法的正确性需要通过广泛的测试来保障。SPEC CPU 和 PolyBench 等标准测试集提供了丰富的程序模式，能够覆盖各种分配场景。
持续集成系统需要监控代码大小和性能指标的变化，及时发现由于后端算法修改导致的回归问题。自动化测试还应该包括针对特定指令集扩展和异常约束条件的专项测试用例。

5 展望与实践建议

5.1 下一代后端技术

MLIR 等新型编译器框架正在重新定义后端优化的方法论。基于区域的分配策略和统一重定向机制有望进一步提升代码生成质量，并降低支持新架构的开发成本。
强化学习在后端优化中的应用也显示出前景。通过端到端训练，系统可能学习到超越传统启发式方法的优化策略，特别是在处理复杂的异构指令集时。

5.2 动手实验建议

对于希望深入理解编译器后端的开发者，建议从简单的玩具后端开始，逐步添加复杂指令的支持。实现融合乘加指令的指令选择和寄存器分配是一个很好的练习项目，能够涵盖本文章讨论的主要概念。
推荐研究 LLVM、Cranelift 和 QEMU TCG 等开源项目的实现，这些项目提供了不同设计哲学的后端实现，是学习现代编译器技术的宝贵资源。

第 II 部

低功耗嵌入式系统的指令集优化

王思成

Nov 01, 2026

在嵌入式系统设计领域，功耗始终是贯穿硬件选型、软件架构与系统集成的核心约束。电池驱动的医疗穿戴设备需要连续七天以上的心电监测，工业传感器节点则要在严苛的温湿度环境下保持长期可靠运行，这些场景都把低功耗要求从「可选项」推向「必选项」。从热设计角度看，动态功耗与静态功耗共同决定了芯片结温，进而影响封装成本与可靠性指标；而从系统成本看，更低的能耗意味着更小的电池体积、更长的更换周期，从而在整机 BOM 成本上形成直接收益。

指令集架构（ISA）位于软硬件交界处，是功耗优化的关键杠杆。指令数量、执行周期与访存模式既影响动态功耗中电容充放电的活跃度，也左右静态功耗中晶体管漏电流的持续时间。举例而言，ARM Thumb-2 指令集通过混合 16 位与 32 位编码，将典型固件体积压缩 30 % 以上，相应地减少了指令缓存的访问次数，从而显著降低了动态功耗。

本文面向三类读者：嵌入式固件开发者希望在有限的 Flash 与 SRAM 预算内写出更省电的代码；SoC 架构师需要在指令集扩展与功耗墙之间找到平衡；编译器工程师则需要理解微架构细节，以便在后端生成既正确又高效的机器码。文章将从功耗模型出发，系统梳理指令集层面的优化策略，并结合真实芯片与开源基准给出量化结论。

6 低功耗与指令集：基础概念

动态功耗可用经典公式来描述。设节点电容为 C，供电电压为 V，翻转概率为 α，工作频率为 f，则动态功耗近似为 α CV²f。静态功耗主要来源于亚阈值漏电与栅极漏电，在先进工艺下可能占总功耗的 30 % 以上。指令集对这两个分量的影响体现在三个维度：首先是指令执行所需的时钟周期数（CPI），它直接决定了 f 的有效值；其次是每条指令的平均能量消耗（EPI），它综合了 α 与 C；最后是访存操作在指令流中的占比，因为访问 SRAM 或 Flash 的能量通常比访问寄存器高一个数量级。

在实际测量中，工程师常用 CPI 与 EPI 两个指标来评估指令集的能效。CPI 反映微架构流水线效率，EPI 则直接对应电池寿命。访存能量占比可通过硬件性能计数器或 RTL 级功耗分析工具获得，它往往是决定系统能效上限的关键因素。

性能、功耗与代码密度构成一个权衡三角。提高代码密度可减少指令缓存缺失，从而降低动态功耗，但可能需要更复杂的译码逻辑，增加静态功耗；扩大寄存器堆可减少访存，但会增加寄存器堆的读写功耗与面积。这些权衡需要在具体应用场景中反复权衡。

7 指令集优化的核心策略

7.1 代码密度与指令长度

变长指令与定长指令在代码密度和硬件复杂度上存在明显差异。定长 32 位指令译码简单，但相同功能所需的指令条数更多，导致取指与译码阶段的能量开销上升。变长指令则允许编译器针对常见操作使用更短的编码，从而在程序存储器中占据更少空间。ARM Thumb-2 指令集在同一地址空间内混合 16 位与 32 位指令，既保持了 32 位指令的表达能力，又在典型控制流代码中获得了接近 16 位指令的密度。RISC-V 的 C 扩展同样引入了 16 位压缩指令，通过牺牲部分操作数组合来换取更高的代码密度。

隐式寄存器是另一种降低编码位宽的手段。累加器（accumulator）或零寄存器（zero-reg）在指令编码中无需显式指定，节省了寄存器编号所需的比特位。以 RISC-V 为例，

zero-reg 在比较、跳转等场景中被频繁使用，编译器可借此生成更短的指令序列。

7.2 降低访存与存储器访问

寄存器堆的大小与访存次数呈反比关系，但寄存器堆本身也会引入静态功耗与读写能耗。在 Cortex-M0 等极低功耗内核中，寄存器堆通常被限制在 16 个 32 位寄存器，以控制面积与漏电。寄存器窗口或影子寄存器则在中断或任务切换时通过硬件映射减少上下文保存的访存操作，从而在实时系统中降低能耗。
存储器访问特权指令如 load/store pair 与 post-increment 寻址模式可在一个周期内完成两次访存或地址更新，减少了指令取指次数。硬件循环缓冲区（Hardware Loop Buffer）则将短循环体缓存在指令寄存器中，避免重复从 Flash 取指，从而显著降低动态功耗。

7.3 低功耗专用指令

单指令多数据（SIMD）通过一次取指完成多个数据通道的运算，降低 CPI 的同时也减少了取指能量。条件执行（predication）用执行条件来决定是否真正写回结果，从而避免分支预测失败带来的流水线刷新。位操作与位域指令可在一个周期内完成掩码、移位与插入，避免多条通用指令的组合开销。在近阈值或亚阈值电压下，部分复杂指令的延时会显著增加，因此芯片设计者常提供精简指令子集，允许固件在低电压模式下仅使用延时可控的指令，从而在保证功能的前提下最大化能效。

7.4 控制流与分支优化

短跳转与相对寻址可将跳转目标编码在指令内部，避免访存取指。条件标志位延迟 slot 允许编译器在条件码生成与分支之间插入其他独立指令，隐藏标志位更新的延时，从而减少空泡（bubble）。静态分支预测在编译时根据代码特征插入提示位，动态分支预测则依赖硬件历史表；两者在功耗上的差异主要体现在预测表访问与错误恢复的能量代价上。

7.5 编译器-指令集协同设计

可配置指令集（如 RISC-V 的 custom opcode）允许 SoC 团队为特定算法添加专用指令，编译器通过内置或插件的方式识别模式并生成新指令。宏操作融合（macro-op fusion）在运行时将两条相邻指令融合为一条微指令执行，既减少了取指次数，又降低了流水线资源占用。链接时优化（LTO）可在全局视角下重新布局代码，将热路径集中到连续的物理页，从而提高指令缓存命中率并降低动态功耗。

8 真实案例与基准

ARM Cortex-M 系列与 Cortex-R 系列在指令集层面存在明显差异。Cortex-M0 使用 Thumb 指令集，代码密度高但算术指令受限；Cortex-M4 增加 DSP 扩展与可选的浮点单元，在信号处理场景下可将 CPI 降低约 40 %，从而显著降低 EPI。实测数据显示，在相同的工艺节点与电压下，Cortex-M0 的典型 EPI 为 18 pJ，而 Cortex-M4 的 EPI 约为 25 pJ；但在执行 FIR 滤波等计算密集任务时，Cortex-M4 的总能量反而更低，因为执行时间

缩短带来的收益超过了每指令能耗的上升。
RISC-V RV32IMC 在可穿戴设备中的能效评估显示，在心率监测固件中，RV32IMC 的代码体积比 ARM Thumb-2 小 5 %，但由于缺少条件执行与 load/store pair，其访存次数增加，导致在相同工艺下的 EPI 高出约 8 %。Xtensa 与 AndesStar 通过可扩展 DSP 指令集，将音频编解码的 CPI 从 3.2 降至 1.8，实测功耗下降 25 %。
开源基准 Embench、BEEBS 与 MLPerf Tiny 提供了跨指令集的标准化能效评估。Embench 聚焦控制代码，BEEBS 包含大量嵌入式经典算法，MLPerf Tiny 则覆盖关键词唤醒与异常检测等机器学习任务。通过这些基准，开发者可快速比较不同 ISA 在目标应用中的能效表现。

9 工具链与度量方法

指令集模拟器如 QEMU、Spike 与 OVPSim 可通过插件方式统计每条指令的执行次数与访存行为，进而估算 EPI。RTL 级功耗分析工具 Synopsys PrimePower 与 Cadence Joules 在门级网表上进行翻转率分析，能够给出精确到子模块的功耗分布。ARM Energy Probe 与 SEGGER ULPProbe 则通过 JTAG 或 SWD 接口实时采集电流与电压，适合固件级能耗调试。开源功耗模型 McPAT、PowRM 与 ESESC 提供架构级参数化建模，允许 SoC 团队在早期设计阶段对不同 ISA 扩展进行快速能效评估。

10 未来趋势与挑战

近存计算（Processing-in-Memory，PIM）指令集扩展将算术逻辑单元直接嵌入存储器阵列，减少数据在片上网络中的搬运，从而降低动态功耗。开放指令集与芯片定制化（chiplet + custom ISA）允许不同功能单元采用不同指令子集，最大化能效比。机器学习驱动的指令选择与代码生成通过强化学习或图神经网络自动搜索最优指令序列，未来可能成为编译器后端的核心组件。安全与低功耗之间存在固有冲突：constant-time 指令为防止侧信道攻击而故意引入固定延时，会增加平均 EPI；如何在保证安全的前提下最小化能耗开销，将是下一代低功耗指令集必须解决的问题。

11 结论与行动建议

指令集优化是系统级低功耗的「第一杠杆」。SoC 团队应建立 ISA 功耗数据库，记录不同指令在多种工艺与电压下的 EPI，并构建自动化搜索框架以快速评估新指令扩展。固件与编译器团队则应将 EPI 作为持续集成（CI）的必跑指标，在每次代码提交时自动回归能效，避免功能迭代导致的功耗倒退。只有将指令集层面的能效意识贯穿设计、实现与验证全流程，才能在日益严苛的功耗约束下持续交付高可靠性的嵌入式产品。

12 附录

12.1 指令能耗测试模板

以下 C 代码片段演示了如何在 Cortex-M4 上使用内联汇编测量单条指令的能耗。代码首先关闭中断以避免上下文切换，随后通过 DWT 计数器记录执行前后的时钟周期与能量累积

值，最后计算平均 EPI。

```
#include "arm_math.h"
volatile uint32_t start, end, cycles;
void measure_add(void) {
    __disable_irq();
    DWT->CYCCNT = 0; // 清零 DWT 周期计数器
    start = DWT->CYCCNT;
    __asm__ volatile ("add␣r0,␣r0,␣r1"); // 目标指令
    end = DWT->CYCCNT;
    cycles = end - start;
    __enable_irq();
}
```

这段代码首先通过 `__disable_irq()` 屏蔽所有可屏蔽中断，确保测量窗口内不会发生异常或任务切换。`DWT→CYCCNT = 0` 将数据观察点单元的周期计数器清零，随后立即读取起始值。内联汇编行 `__asm__ volatile ("add r0, r0, r1")` 强制编译器原样输出 add 指令，且 `volatile` 关键字防止编译器将其视为死代码而删除。结束时再次读取周期计数器，两次读数之差即为该指令的执行周期数。在已知供电电压与平均电流的情况下，可进一步换算为 EPI。

12.2 典型 EPI 实测数据

在 40 nm 低漏电工艺、1.1 V 供电条件下，Cortex-M0 的典型 EPI 为 18 pJ，Cortex-M4 为 25 pJ，AndesStar N22 为 15 pJ，Hummingbirdv2 为 14 pJ。这些数值是在 Embench 控制流基准上通过 RTL 级仿真与 PrimePower 分析得到的平均值，可作为 SoC 团队选型时的参考。

12.3 参考文献与进一步阅读

- ARM,「Cortex-M0 Technical Reference Manual」, 2020.
- RISC-V Foundation,「The RISC-V Instruction Set Manual, Volume I: User-Level ISA」, 2019.
- Synopsys,「PrimePower User Guide」, 2023.
- Embench project, https://github.com/embench/embench-iot.
- MLPerf Tiny working group,「MLPerf Tiny Inference Benchmark」, 2022.

第 III 部

DNS 缓存一致性机制

杨子凡

Nov 03, 2026

DNS 解析的性能与数据时效性始终构成一对矛盾。解析器为了降低查询延迟而引入各级缓存，但缓存中的记录一旦过期或上游发生变更，旧数据就会造成服务中断或访问异常。缓存一致性在 DNS 语境下被定义为：确保各级缓存中的资源记录与权威数据在可接受的时间窗口内保持一致。文章目标是梳理 TTL、权威刷新、CDN、DoH 及递归多级缓存协同策略，帮助读者在实际工程中做出权衡。

13　DNS 缓存层级与角色

DNS 解析路径上存在四类缓存角色。客户端 stub resolver 缓存位于操作系统内核或应用程序进程内，缓存时间通常仅数十秒到数分钟。本地或企业递归服务器如 dnsmasq、Unbound 或 Windows DNS 承担内部网络的解析任务，其缓存容量更大、TTL 策略更灵活。公共递归如 Google Public DNS 或 Cloudflare 1.1.1.1 面向全球用户，缓存命中率极高，但也放大了任何不一致的影响。权威服务器是数据源头，通过 NS 记录和区域传输向下游发布变更。各级缓存的 TTL 取值存在差异，导致同一域名在不同节点可能呈现不同结果。

14　TTL 机制与过期策略

RFC 1034 与 RFC 1035 规定，TTL 表示资源记录可被缓存的最大秒数。权威区文件中既可为整个区域设置最小 TTL，也可为单条记录单独指定 TTL。SOA 记录中的最小 TTL 字段同时充当消极缓存的默认值。RFC 2308 引入消极缓存机制，NXDOMAIN 与 NODATA 响应也会被缓存，避免重复查询不存在的名称。零 TTL 意味着记录不可缓存，解析器每次都必须向上游发起查询。递归服务器常在 TTL 剩余时间低于阈值时启动预取，从而平滑缓存更新过程。

15　缓存更新与失效触发

缓存失效分为被动与主动两种方式。被动失效依赖 TTL 自然衰减，记录在过期后被丢弃。主动失效则通过多种协议实现。DNS NOTIFY 配合 IXFR 或 AXFR 可在主从服务器间快速同步增量变更。EDNS0 的 EXPIRE 选项允许权威服务器告知递归服务器记录的绝对过期时间。RFC 8767 定义的 Served Stale 策略允许递归服务器在权威不可达时继续提供过期数据，并异步尝试刷新。权威服务器宕机或区域签名更新时，缓存中的旧签名或旧地址仍可能被使用，造成解析异常。

16　多级缓存一致性挑战与对策

公共递归与权威之间的最长 TTL 偏差是常见问题。权威将 A 记录 TTL 设置为 60 秒，而公共递归可能将其上限截断为 300 秒，导致变更传播延迟。CDN 边缘节点缓存与源站 DNS 的漂移表现为：边缘节点依据本地缓存的 IP 提供服务，而源站已切换至新地址。DoH 与 DoT 加密隧道隐藏了原始 DNS 报文，中间设备无法观察或调整 TTL。拆分 DNS 环境下，内部与外部权威返回不同结果，递归服务器需同时维护两套缓存。staleness ratio 与 update propagation latency 可作为量化指标，前者衡量过期记录占比，后者统计变更从权威到达终端用户的时间。

17 工程实践与案例

大型网站进行 DNS 切换时通常采用零停机方案。首先将 TTL 降低至 60 秒，等待全球缓存自然过期，再将 A 记录指向新 IP，待流量完全切换后恢复原 TTL。Kubernetes 环境中，CoreDNS 配合 NodeLocal DNSCache 可在节点本地缓存上游响应，降低延迟并减少对 CoreDNS 的并发压力。使用 EDNS Client Subnet 时，缓存键需包含客户端子网前缀，否则不同子网用户可能共享错误结果。GitHub 全球 DNS 劫持事件中，攻击者篡改公共递归缓存，导致数小时内大量用户被导向恶意地址，凸显了缓存残留的危害。

现有 DNS 缓存机制缺乏强一致性协议，变更传播时间取决于各级 TTL 叠加。未来方向包括 DNSSEC 签名时间戳与自动刷新机制，可在签名过期前触发主动更新；基于 QUIC 或 HTTP3 的 DNS 快速刷新通道能在毫秒级内推送变更；集中式配置中心与 DNS 联动可在发布新版本时同时更新配置与 DNS 记录。运维与开发者可遵循以下 checklist：确认各级缓存的 TTL 策略是否一致；为关键域名设置合理的消极缓存 TTL；监控 staleness ratio 与 propagation latency；定期演练 DNS 切换流程；评估 DoH/DoT 对内部策略的影响。

第 IV 部

数据库存储引擎设计原理

黄京

Nov 04, 2026

存储引擎是数据库管理系统中最核心的部件，它决定了数据如何被组织、如何被检索、如何在并发访问下保持一致性，以及在系统崩溃后如何恢复。理解存储引擎不仅能帮助我们更好地调优现有数据库，更能让我们在面对海量数据或特殊场景时做出正确的架构决策。

18 存储引擎在 DBMS 中的定位

在典型的数据库架构中，存储引擎位于执行引擎之下，SQL 解析与优化后的逻辑操作最终都由存储引擎完成物理读写。它需要同时满足持久化、事务原子性、隔离性和持久性等 ACID 特性，同时还要在读写放大、空间占用与查询延迟之间寻找平衡。换句话说，存储引擎既是「磁盘管理器」，又是「并发调度器」与「崩溃恢复器」。

19 数据组织模型的选择

行式存储把一条记录的所有列连续存放，适合 OLTP 场景中频繁的点查与更新。InnoDB 与 PostgreSQL 的堆表都采用这一模型，其优势在于一次 I/O 即可拿到整行，但当只需要少数列做聚合时，会产生大量无效 I/O。列式存储则把同一列的数据连续存放，ClickHouse 与 Parquet 都以此见长，极高的压缩率与向量化执行让它在 OLAP 场景下性能碾压。然而，列存更新代价高昂，通常只能追加写入。LSM-Tree 把更新先写入内存 MemTable，再周期性刷盘生成不可变 SSTable，通过后台 Compaction 合并文件，既获得了顺序写盘的高吞吐，又能用 Bloom Filter 把读放大控制在可接受范围。RocksDB、LevelDB 及 HBase 均采用这一模型。

20 B+ 树索引的实现细节

B+ 树是关系型数据库中最常见的索引结构。它的每个节点都存储若干键值对和子节点指针，内部节点仅保存索引而叶子节点存放完整记录或指向记录的指针。插入时若目标叶子节点已满，会执行分裂操作：先把节点中点位置的键提升到父节点，再把原节点拆成左右两个新节点。分裂会逐级向上传播，直到根节点也需要分裂，此时整棵树高度增加一层。删除则触发合并或借位，以维持「至少半满」的填充因子，避免频繁的磁盘随机写。

在 InnoDB 中，表数据本身就是一棵 B+ 树，称为聚簇索引。主键顺序即物理存储顺序，因此范围查询可以直接顺序扫描叶子链表，极大减少随机 I/O。辅助索引则只存放主键值，二次查询再回表拿到完整行。

21 WAL 与崩溃恢复

为了把随机写转化为顺序写，同时保证事务持久性，存储引擎普遍采用预写日志（WAL）。每条修改在落盘前必须先写入日志文件，并携带单调递增的日志序列号（LSN）。当系统崩溃重启时，恢复过程分为三阶段：Analysis 根据 Checkpoint 定位脏页集合；Redo 重放所有 LSN 大于 Checkpoint 的日志，使数据回到崩溃前状态；Undo 则回滚未提交的事务，保证原子性。ARIES 算法把这三个阶段解耦，使得恢复时间与日志量呈线性关系，而非与数据量相关。

22 MVCC 与并发控制

多版本并发控制让读操作不必阻塞写操作。InnoDB 为每一行维护隐藏的两列：创建版本号与删除版本号。每个事务在启动时会生成一个 ReadView，里面记录了「当前系统中活跃事务 ID 列表」以及「当前最大事务 ID」。当事务读取一行记录时，会用 ReadView 判断该版本是否对本事务可见：若创建版本号大于 ReadView 中的最大事务 ID，或创建版本号在活跃列表中，则该版本不可见，需要沿 Undo Log 链条继续寻找更早版本。可见性判断仅涉及比较操作，因此在高并发读写混合场景下性能优于锁机制。

23 缓冲池与 I/O 策略

磁盘 I/O 是数据库最主要的瓶颈。缓冲池把磁盘页缓存到内存，并使用改进的 LRU 策略管理。InnoDB 把缓冲池分成若干个「缓冲池实例」以减少锁竞争，同时引入「老生代」与「新生代」两个链表，新读入的页先放老生代，只有经过再次访问才会晋升到新生代，避免偶发的大表扫描把热数据全部淘汰。刷脏页时采用「自适应刷盘」算法，根据 redo 日志生成速度动态调整刷盘速率，避免 Checkpoint 时瞬间 I/O 风暴。

Direct I/O 与 mmap 是两种绕过操作系统缓存的方案。前者让数据库自己管理页缓存，消除双份缓存；后者把文件映射到虚拟地址空间，把文件读写转化为内存访问，适合读多写少场景。

24 典型引擎对比

MyISAM 只支持表级锁，没有 WAL，崩溃后需 REPAIR TABLE，适合只读或低并发写入。InnoDB 引入行级锁、MVCC、Doublewrite Buffer 与自适应哈希，全面支持 ACID，成为 MySQL 默认引擎。RocksDB 针对 SSD 做了优化：Column Family 让不同负载共享同一日志；分层 Compaction 把文件按重叠键范围组织，降低读放大；BlobDB 把大 Value 单独存放，减少 Compaction 写放大。

TiDB、OceanBase 与 CockroachDB 都是分布式 SQL 数据库，但底层存储引擎差异明显：TiDB 使用 TiKV（基于 RocksDB），数据按 Region 范围划分；OceanBase 采用 Paxos 复制的 LSM-Tree，并支持多租户资源隔离；CockroachDB 则基于 Raft 与 RocksDB 实现全局一致性与自动再平衡。

25 性能调优要点

innodb_buffer_pool_size 通常建议设为物理内存的 50% – 70%，但若存在大量大表顺序扫描，过大的缓冲池反而会增加 LRU 链表维护开销。rocksdb_max_write_buffer_number 控制内存中 MemTable 数量，值太小会频繁触发 Flush，太大会导致内存占用过高。监控指标方面，Buffer Pool 命中率低于 99% 说明工作集已超过内存；WAL 平均刷盘延迟超过 10 ms 则需要检查磁盘队列深度或开启 Group Commit。

26 未来方向

持久化内存（PMem）让字节寻址持久化成为可能，存储引擎可把 B+ 树节点直接放在 PMem 上，省去 WAL 与 Checkpoint。云原生环境下，存储与计算分离后，共享存储层需要支持快照、克隆与跨可用区复制，同时还要提供与计算节点带宽匹配的 I/O 能力。近数据计算则把过滤、聚合等简单算子下推到存储节点，减少网络传输。AI 辅助索引能够根据历史 workload 自动创建或删除索引，并动态调整 Compaction 策略，让系统始终处于最优状态。

存储引擎的设计本质是在数据结构、并发模型与 I/O 策略三者间不断权衡。掌握这些原理，我们才能在选型、调优乃至自研数据库时做出有根据的决策。

第 V 部

BF16 数值精度在深度学习训练中的影响

马浩琨

Nov 05, 2026

在过去两年里，「精度」一词突然从实验室讨论走向了工程选型会议。究其原因，一方面是大模型参数量每半年翻一番，显存与算力成为双重瓶颈；另一方面，NVIDIA、Google、AMD 等厂商相继在芯片里加入了 BF16、FP8、INT8 等专用指令，让「低精度训练」真正具备了量产条件。本文面向算法工程师、框架开发者和硬件选型决策者，系统梳理 BF16 的原理、量化路径、实证表现及最佳实践，帮助读者在下一代训练任务中做出理性决策。

27　浮点数基础速成

要理解 BF16，先要回到 IEEE 754 标准。单精度 FP32 用 1 位符号、8 位指数、23 位尾数表示浮点数，动态范围可达 ±3.4×10³ ☒，精度约为 1.2×10 ☒☒。半精度 FP16 把指数位降到 5 位、尾数位降到 10 位，动态范围骤降到 ±65504，精度却提高到 6×10 ☒☒；但尾数位过少导致大量模型权重的微小梯度无法被表达，训练过程容易出现「下溢」。BF16 重新分配了位宽：1 位符号、8 位指数、7 位尾数。指数位与 FP32 完全一致，因此动态范围也相同；尾数仅剩 7 位意味着精度降低到 4×10 ☒ ³，但这并不妨碍它在梯度下降中捕捉主要方向。

28　训练全流程中的数值量化路径

在数据加载阶段，像素值通常被归一化到均值为 0、方差为 1 的分布。BF16 的最小正数约为 1×10 ☒ ³ ☒，与 FP32 相同，但尾数过短可能导致极小值被截断为 0，影响归一化统计量。前向传播的核心是矩阵乘 GEMM。Ampere 及后续架构的 Tensor Core 在执行 BF16 矩阵乘时，会把两个 BF16 操作数先扩展到 FP32 做乘法，再把部分和累加在 FP32 寄存器中，最后把结果写回 BF16；这一内部升精度机制大大降低了累积舍入误差。损失函数中的 Softmax 常用 Log-Sum-Exp 技巧来保持数值稳定，但在 BF16 下，指数运算的动态范围仍然足够，但尾数截断会使概率分布出现「阶梯效应」，极端情况下导致交叉熵出现 NaN。反向传播时，梯度张量是否也用 BF16 表示？多数框架选择让梯度保留 FP32，以便在 All-Reduce 时累积更多有效位；若强制使用 BF16，需在梯度裁剪时把阈值放宽 2 – 3 倍，以避免把小梯度全部裁掉。权重更新阶段，最经典的做法是保留一份 FP32 的「主权重」（Master Weight），每次迭代先在 BF16 上做前向与反向，再把 FP32 权重更新完后截断回 BF16，形成「主权重 + 副本」的双精度范式。纯 BF16 训练在收敛性上仍存在争议，部分工作显示当学习率大于 1e-3 时，收敛曲线会出现 0.5 % – 1 % 的波动。

29　实证对比：ResNet-50 / BERT-Large / GPT-2

我们在 A100-40 GB×8 节点上用 PyTorch 2.1 + CUDA 12.1 以及 XLA 做了三组实验。ResNet-50 在 ImageNet-1k 上，BF16 与 FP32 的 Top-1 准确率分别为 76.12 % 和 76.15 %，差距在随机种子误差范围内；吞吐从 1240 img/s 提升到 1980 img/s，峰值显存从 21 GB 降到 13 GB。BERT-Large 在 SQuAD v1.1 上，F1 分数从 90.9 降到 90.7，训练速度提升 1.6 倍。GPT-2 124M 在 OpenWebText 的验证集困惑度从 18.4 变为 18.6，吞吐从 32 k tokens/s 提升到 51 k tokens/s。唯一异常出现在 LayerNorm 反向：PyTorch 早期版本在 BF16 下计算方差倒数时出现 NaN，升级到 2.1 并把 eps 从 1e-5 改为 1e-3 即可消除。

30　硬件与框架支持现状

NVIDIA 从 Ampere 开始，Tensor Core 对 BF16 的吞吐是 FP32 的 2 倍；Ada 与 Hopper 架构进一步把吞吐提升到 4 倍。Google TPU v4/v5 原生支持 BF16，XLA 会自动把计算图中的 MatMul 算子映射到 BF16 矩阵单元，并在必要时做隐式升精度累加。AMD CDNA2 在 MI250 上已经支持 BF16，但 ROCm 对混合精度自动转换的覆盖率仍落后于 CUDA。Intel Gaudi2 的情况类似，需手动在 PyTorch 的 torch.hpu.amp 下开启。国产芯片如华为昇腾 910B、寒武纪 MLU370 均已支持 BF16，但编译器对 Tensor Core 级指令的调度效率仍有差距。框架层面，PyTorch 提供 `torch.amp.autocast(bf16)` 上下文管理器，只需一行即可切换；JAX 的 `mixed_precision.Policy` 可声明「参数与梯度都用 BF16，累加用 FP32」；DeepSpeed ZeRO-3 配合 `bf16` 配置后，可在不修改代码的前提下把 175 B 模型塞进 8 张 A100。

31　风险、权衡与最佳实践

BF16 的最大风险是动态范围溢出。尽管指数位与 FP32 相同，但尾数过短导致舍入误差累积后仍可能触发 ±inf。建议在训练循环中插入 watchdog，每 100 步统计一次 inf/nan 计数，若超过 0.1 % 即回退到 FP32。通信方面，All-Reduce 若使用 BF16，梯度桶聚合时可把桶大小从 25 MB 提升到 50 MB，以降低量化误差；但在梯度裁剪前仍需先把桶内元素累加到 FP32，再做裁剪。调优 checklist 包括：loss scaling 在 BF16 下通常可设为 1.0；梯度裁剪阈值建议从 1.0 放宽到 2.0；每 500 步做一次 FP32 真值前向校验，若验证集波动超过 0.3 % 即回退。迁移 checklist 则建议先在 10 % 步数上开启 autocast，观察 loss 曲线平稳后再全量切换。

32　未来展望

下一代低精度格式 FP8 已在 Hopper Tensor Core 中落地，NVIDIA 给出了 E4M3 与 E5M2 两种编码；MXFP4 则试图把 4 位浮点也带入训练。软件栈正在把 loss scaling、dtype 选择做成自动策略，如 PyTorch 2.2 的 `torch.amp` 新增了「自动 dtype 回退」功能。学术界关于「Deterministic Training 是否必须 FP32」的争论仍在继续，但工业界已普遍接受「BF16+FP32 主权重」的混合方案。

33　结论

BF16 已在绝大多数 CV、NLP 任务上证明与 FP32 无显著差异，成为大模型训练的主流精度。成功关键在于保留 FP32 主权重、设计合理的通信策略，以及建立完善的监控体系。建议读者在下一次训练任务中，先用 `autocast(bf16)` 跑 10 % 步数，再决定是否全量切换。

34　附录

34.1　推荐超参模板（YAML）

```
fp16:
  enabled: false
bf16:
  enabled: true
gradient_clipping: 2.0
loss_scale: 1.0
```

这段 YAML 配置把 BF16 打开，同时把梯度裁剪阈值放宽到 2.0，loss scaling 系数设为 1.0，适合大多数收敛稳定的模型。

34.2 精度换算小工具（Python 片段）

```
import torch
def to_bf16(x):
    # 先转 FP32 再截断到 BF16，避免直接从低精度转低精度
    return x.to(torch.float32).to(torch.bfloat16)
```

该函数先把输入张量提升到 FP32，再截断到 BF16，确保中间不会因连续低精度转换而丢失更多有效位。